



AUBURN

UNIVERSITY

Introduction to Operating Systems

Overview

Objectives

- Review/tour the key components of a **Computer System**
- Learn about and understand an **operating system**
- Learn and understand the **role/duties of an operating system**:
 - **Processes** Management
 - **Memory** management
 - (External) **Storage** Management
 - **I/O** System
 - **Protection and Security**

Requirements

- Passing grade in COMP 2710 and (COMP 3350 or ELEC 2220)
- Complete assigned readings



Computer System Review

- **What** is an Operating System?
- **What** is a Computer System?
- **Why** would a Comp. System need an Operating System?
- **Key mechanisms** that help an OS fulfill its mission
 - Interrupts
 - Direct Memory Access (DMA)

What is an Operating System (OS)?

❓ An OS is a **PROGRAM (piece of software)** that makes the use of a **computer system** convenient, secure, and efficient.

❓ This definition raises some questions:

❓ **What** is a computer system?

❓ **Why** would we need an OS?

❓ (Can't a computer system and its user(s) take care of themselves?)

❓ What does "**efficient**" mean?

- OS is a kind of government
- Efficient would mean that use the CPU for not too long & for a certain task. It has competing needs

What is a Computer System?

? Computer system is made of four major components:

? **Hardware (Computer)** – provides basic computing resources

? CPU, memory, I/O devices

? **Operating system**

? Controls, coordinates, and optimizes use of hardware among various applications and users
-optimizing for applications (browsers, compilers)

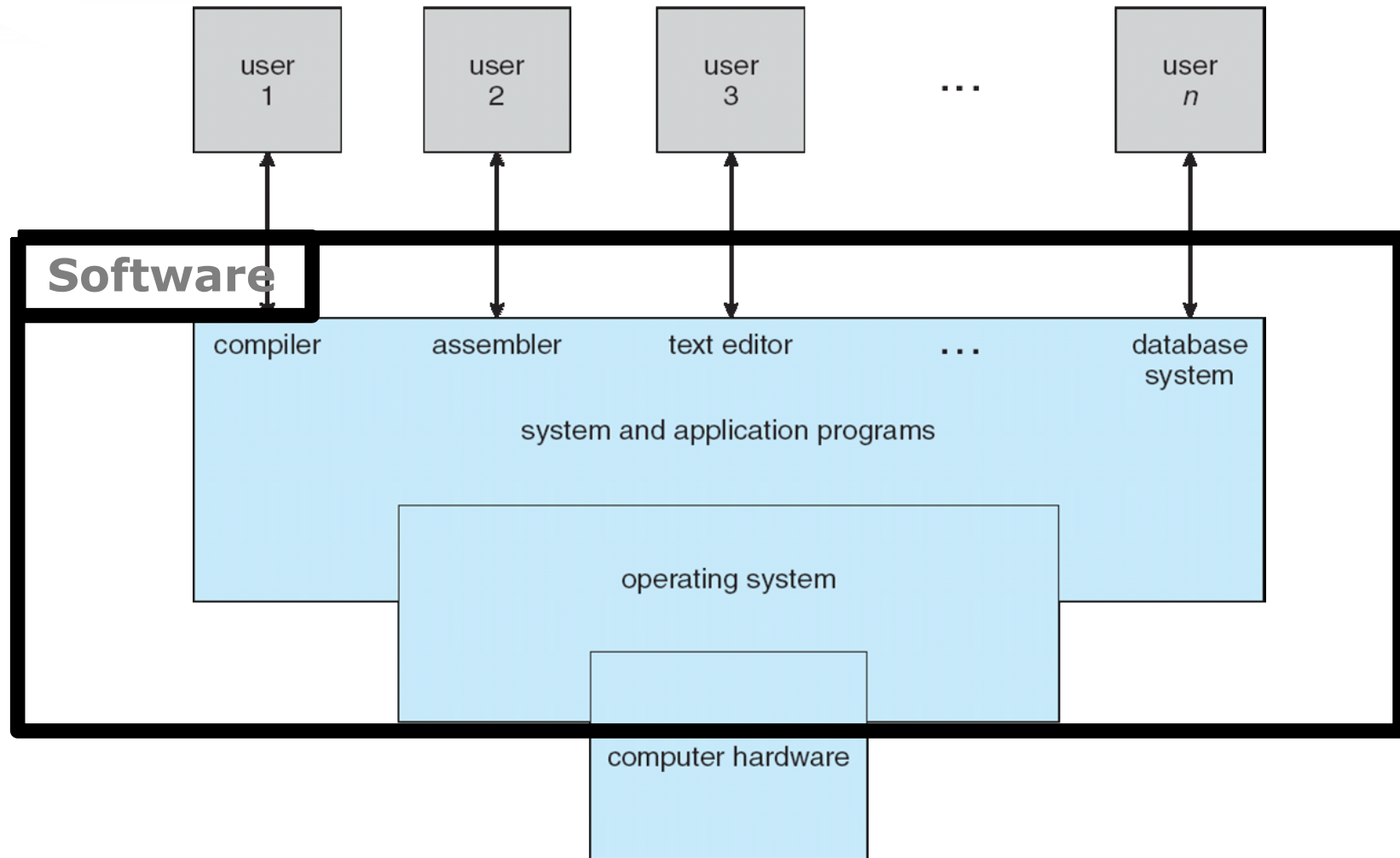
? **Application programs** – define the ways in which the system resources are used to solve the computing problems of the users

? Word processors, compilers, web browsers, database systems, video games

? **Users**

? People, machines, other computers

Four Components of a Computer System



A Computer Is a **Von Neumann Machine**

- ❑ A **Von Neumann Machine** (**Architecture**) consists of a machine with:
 - ❑ A **processing unit** containing a logic and arithmetic unit (ALU) and registers (storage): particularly an **instruction register** and a **program counter register**
 - ❑ A **memory system** that stores instructions (program) and data
 - ❑ An external **mass storage**
 - ❑ An **input/output system**

- ❑ All computer-based devices (desktop, laptops, cellphones, cameras, washing machines....) have the Von Neuman Architecture (known also as the **Princeton Architecture**)

- ❑ A CPU spends all its time:
 - ❑ **Reading** instructions or data (operands)
 - OR**
 - ❑ **Writing** back results/data

Where does hardware meet the Operating System?

? bootstrap program is loaded at power-up or reboot

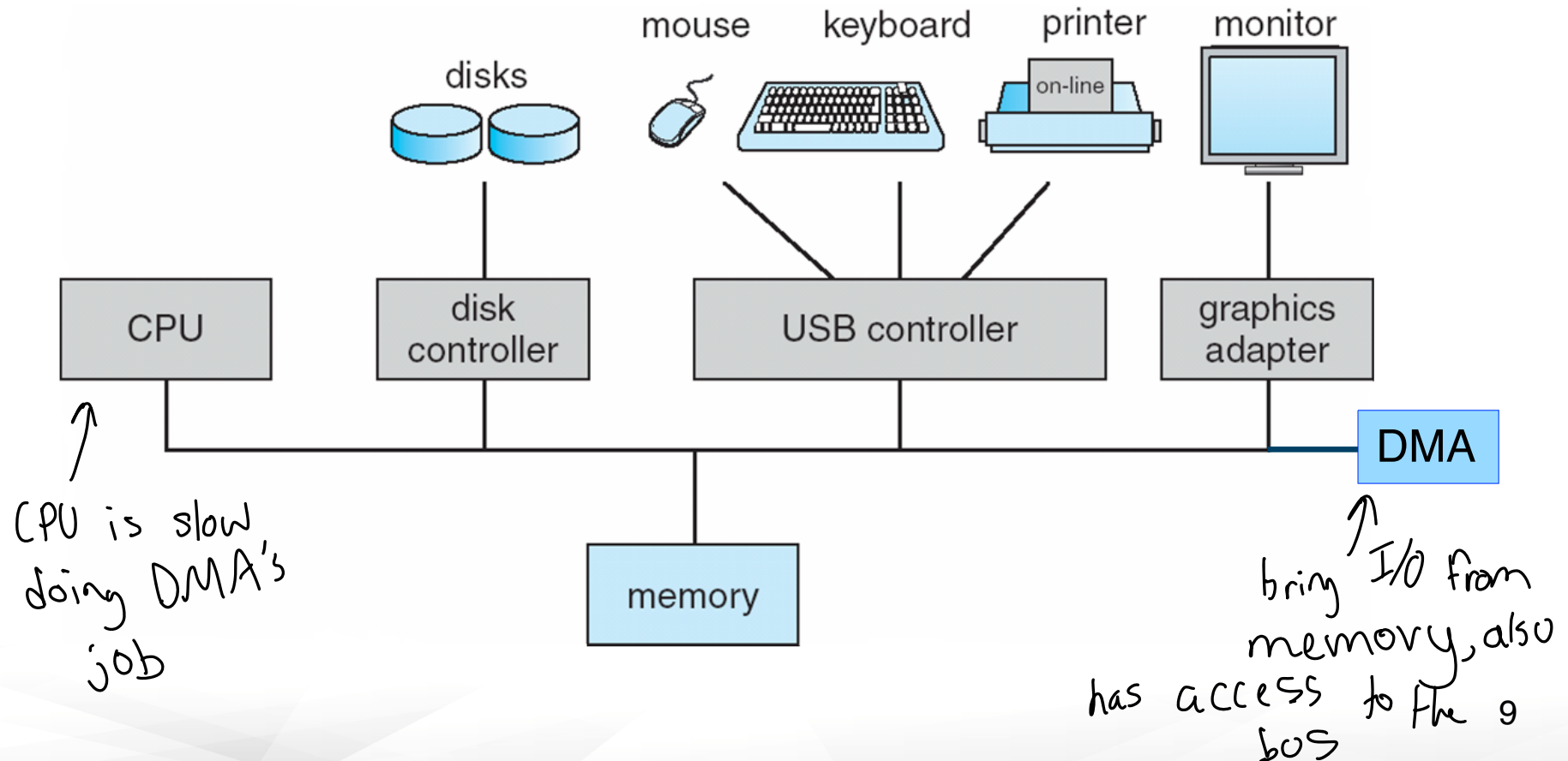
- ? Typically stored in ROM or EPROM, generally known as **firmware**, instruction in the firmware
- ? Initializes all aspects of system
- ? Loads operating system **kernel** and starts execution

hardware memory, turn off & it stays

Computer System Organization

? Computer-system operation

- ? One or more CPUs, device controllers connect through common bus providing access to shared memory
- ? **Concurrent** execution of CPUs and devices competing for memory cycles



Computer-System Operation

- ❑ I/O devices and the CPU can execute **concurrently**
- ❑ Each device controller is in charge of a particular device type
- ❑ Each device controller has a **local buffer**, just memory accessible to DMA or CPU
- ❑ CPU **directs** data from/to main memory to/from local buffers
- ❑ Device controller informs CPU that it has finished its operation by causing an **interrupt**
interrupt - triggers an interrupt in hardware, OS is driven by interrupt

Why would a Comp. System need an Operating System?

?Answer: Input/Outputs are very slow in comparison the CPU speed

- ? CPU is not well suited from transferring data from the main memory to the I/O system (display, mass storage, network.....)
- ? I/O system is in general slow compared to the CPU.
- ? I/O operations are NECESSARY and occur often (get new data to process (Input) and deliver new results).

?Questions:

1. Should the CPU be used to transfer data between memory and I/O system? *NO*
2. If not,
 - a) who should perform the transfers? *DMA*
 - b) Since the CPU is not used during the data transfers, what to do with the CPU during these data transfers? *Look for something else*

A key (Important) Mechanism : Interrupts

? What is an **interrupt**?

? In short, a mechanism to **call a routine** (method) using an external **hardware event** (key stroke, network packet arrival, mouse click, inserting a USB device, ...)

? What for?

? **Response time**: address immediately users/devices requests/events

? How does it work?

? During the initialization, an interrupt routine handler (**IRH**) is associated/mapped with an event.

? When the event occurs,

? the CPU saves the current program counter (to return later) as well as other registers

? the CPU uses that mapping to determine which IRH to call.

? The IRH is executed

? Registers are restored

? The program counter is restored (the CPU returns to the interrupted thread...).

? An operating system is essentially **interrupt-driven**.

? A trap is a software interrupt triggered by user's request, faults, or exceptions.

Another key (Important Mechanism): **Direct Memory Access**

- DMA (waiter analogy), cook is CPU

❓ **DMA:** A **critical/important** hardware component

❓ DMA is a hardware device that **transfer blocks** of data between the I/O buffers and the main memory.

❓ **How does DMA work?**

❓ The OS will provide as inputs the source address **SA**, the destination address **DA**, and the length **L** of the block to transfer. DMA will copy L bytes from source address S to destination address DA.

❓ When the DMA completes the transfer, it interrupts the OS.

- DMA has 2 counters (1 & 2), one takes address from memory & 1 gives address to I/O

❓ **Benefits:**

❓ DMA transfers can be completed **concurrently** with CPU operations

❓ Device controller transfers blocks of data from buffer storage directly to main memory **without CPU** intervention

❓ Only one interrupt is generated per block, rather than one interrupt per byte

Operating System Overview

- **Questions:**
 1. Should the CPU be used to transfer data between memory and I/O system? *No*
 2. If not,
 - a) who should perform the transfers? *DMA*
 - b) Since the CPU is not used during the data transfers, what to do with the CPU during these data transfers? *calculations*
- **What** is an Operating System?
- **Role/functions/Services** of an OS
 - Process Management
 - Memory Management
 - Storage Management
 - I/O System
 - Protection and Security

Why is an OS needed?

?Questions:

1. Should the CPU be used to transfer data between memory and I/O system?
 1. **No**
 2. It would be **time inefficient** (**each** transfer involves **reading** multiple instructions, **reading** the source, **writing** the destination, **updating** pointers to source and destination)
 3. Transfer is a very simple operation that can be handled by **hardware** using the **DMA** device.
2. If not,
 - a) who should perform the transfers?
DMA
 - b) Since the CPU is not used during the data transfers, what to do with the CPU during these data transfers?

We could have **many** other **users** or other **applications** “in the system” with the hope that **AT LEAST** one of them is **ready** to use the CPU (i.e., it is not performing I/O operations)
3. If we have many users or applications “in the system” then we must **share** the computer system **resources** among them **efficiently and fairly**.

So, What IS an Operating System?

❓ OS is a program: a piece of **software**.

❓ OS is a **resource allocator**

❓ Manages all resources (**CPU**, **memory system**, **I/O**)

❓ Decides between conflicting requests for efficient and fair resource use

❓ OS is a **control program**

❓ Controls execution of programs to prevent errors and improper use of the computer

❓ **Remarks:**

❓ No universally accepted definition

❓ “Everything a vendor ships when you order an operating system” is good approximation, but varies wildly

❓ “The one program running at all times on the computer” is the **kernel**. Everything else is either a system program (ships with the operating system) or an application program.

Resources To Manage

? **CPU** to be shared among

? users?

? “applications”?

? processes ?

? or threads?

? **Memory System** to be shared

? within a process

- finite
? **I/O System**

“Keeping Many In the System”

Multiprogramming

batch prog.

time sharing

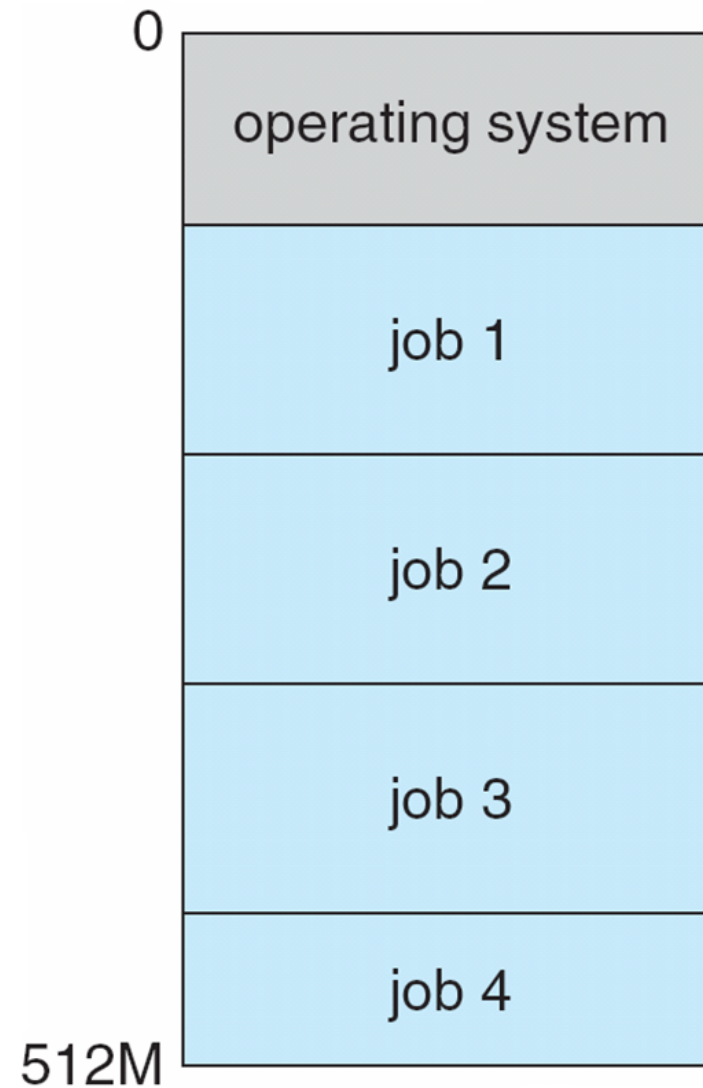
? **Multiprogramming** needed for efficiency

- ? Single user cannot keep CPU and I/O devices busy at all times
- ? Multiprogramming organizes jobs (code and data) so CPU always has one to execute
- ? A subset of total jobs in system is kept in memory
- ? One job selected and run via **job scheduling**
- ? When it has to wait (for I/O for example), OS switches to another job

? **Timesharing (multitasking)** is logical extension in which CPU switches jobs so frequently that users can interact with each job while it is running, creating **interactive** computing

- ? **Response time** should be < 1 second
- ? Each user has at least one program executing in memory → **process**
- ? If several jobs ready to run at the same time → **CPU scheduling**
- ? If processes don't fit in memory, **swapping** moves them in and out to run
- ? **Virtual memory** allows execution of processes not completely in memory

Memory Layout for Multiprogrammed System



Can't run at the same time but run concurrently

Memory System

- Memory System: an **Hierarchy**.
- **Why** is the memory system **hierarchical**?
- **Role** of the OS to manage/move data in the memory system



Memory System

? **Registers**

? **Cache**

? **Main memory** – only large storage media that the CPU can access directly

? **Random access**

? Typically **volatile**

? **Secondary storage** – extension of main memory that provides *larger* **nonvolatile** storage capacity

? **Magnetic disks** – rigid metal or glass platters covered with magnetic recording material

? Disk surface is logically divided into **tracks**, which are subdivided into **sectors**

? The **disk controller** determines the logical interaction between the device and the computer

? **Network (clouds)**

Memory System Hierarchy

? Storage systems organized in an hierarchy because of

? Speed

? Cost

? Volatility ← some memory, turn off & still have data

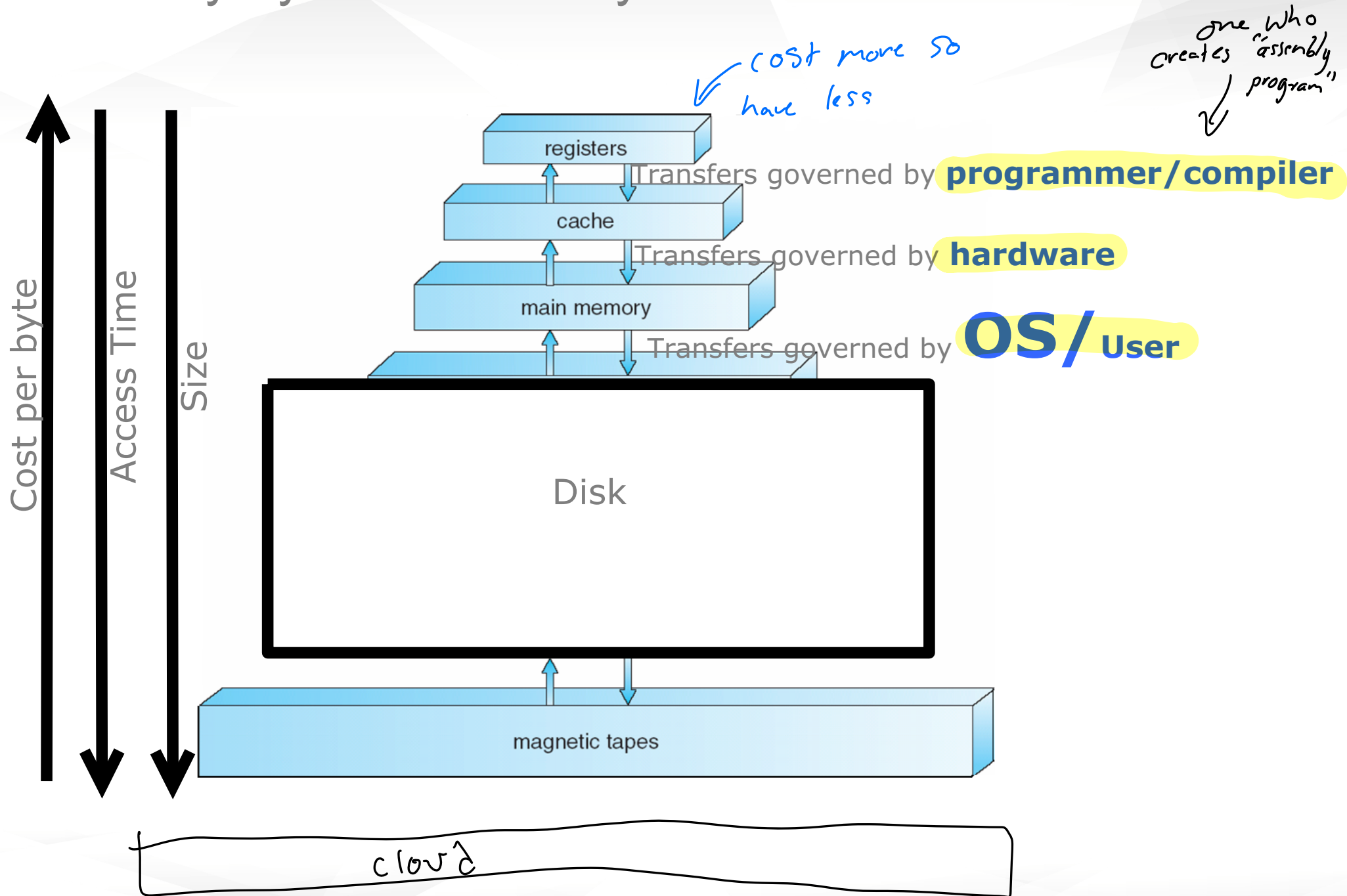
Faster costs more.

They start with the budget

? **Caching** – copying information into faster storage system; main memory can be viewed as a *cache* for secondary storage

bring item from a level to a next level that is faster

Memory System Hierarchy



Performance of Various Levels of Storage

☐ Movement between levels of storage hierarchy can be explicit or implicit

Price	?	?	?	?
Level	1	2	3	4
Name	registers	cache	main memory	disk storage
Typical size	< 1 KB	> 16 MB	> 16 GB	> 100 GB
Implementation technology	custom memory with multiple ports, CMOS	on-chip or off-chip CMOS SRAM	CMOS DRAM	magnetic disk
Access time (ns)	0.25 – 0.5	0.5 – 25	80 – 250	5,000.000
Bandwidth (MB/sec)	20,000 – 100,000	5000 – 10,000	1000 – 5000	20 – 150
Managed by	compiler	hardware	operating system	operating system
Backed by	cache	main memory	disk	CD or tape

ratio time is
about 1000

Caching

- ? The hierarchical memory makes sense ONLY because:
 - 1) Faster costs more
- ? Important principle, performed at many levels in a computer (in hardware, operating system, software)
- ? Information in use copied from slower to faster storage temporarily
- ? Faster storage (cache) checked first to determine if information is there
 - ? If it is, information used directly from the cache (fast)
 - ? If not, data copied to cache and used there
- ? Cache smaller than storage being cached
 - ? Cache management important design problem
 - ? Cache size and replacement policy

Why Does Caching Work?

❓ Caching and the hierarchical memory makes sense ONLY because:

- 1) Faster memory costs more
- 2) Most programs exhibit
 - a) **Temporal** locality: **recently** used items will be used again
 - b) **Spatial** locality: **neighbors** of a recently used item will be used soon

Brief OS Duties Overview

- **Role/functions/Services** of an OS
 - Process Management
 - Memory Management
 - Storage Management
 - I/O System
 - Protection and Security

OS Management Duties

- ? Process Management
- ? Memory Management
- ? Storage Management
- ? I/O subsystem
- ? Protection and Security

? These duties differ depending on the kind of computers

- ? General purpose desktops
- ? Servers
- ? Mobile systems
- ? Embedded systems

Real-time OS (not Windows, iOS, Linux) ← disables interrupts & can't stop it
- control some systems with some deadlines

First, How to Insure that the OS is the **BOSS?**

? **Dual mode** of CPUs

- ? **Kernel** (privileged) mode

- ? **User** mode

? Notion of “**System Call**”

- call from your program to the OS, because OS can give instructions.
- Ex. Kid using parent credit card

Process Management

? A **process** is a program in execution. It is a unit of work within the system. Program is a *passive entity*, process is an **active entity**.

? Process needs resources to accomplish its task

? CPU, memory, I/O, files

? Initialization data

? Process **termination** requires reclaim of any reusable resources

finish on
its own
or kickout

OS does
this

? Typically system has many processes, some user, some operating system running **concurrently** on one or more CPUs

? Concurrency by multiplexing the CPUs among the processes / threads

Process Management Activities

The operating system is responsible for the following activities in connection with process management:

? **Creating** and **deleting** both user and system processes

? **Suspending** and **resuming** processes

? Providing mechanisms for **process synchronization**

? Providing mechanisms for **process communication**

? Providing mechanisms for **deadlock handling**

*do things in the right order
OS provides communication*

Memory Management

❓ Memory management determines **what** is in memory **when** and **where**

❓ Optimizing CPU utilization and computer response to users

❓ Memory management activities

❓ Keeping track of which parts of memory are currently being used and **by whom**

❓ Deciding which processes (or parts thereof) and data to move into and out of memory

❓ Allocating and deallocating memory space as needed

Storage Management

- ❑ OS provides uniform, logical view of information storage
 - ❑ Abstracts physical properties to **logical storage unit** - **file**
 - ❑ Each medium is controlled by device (i.e., disk drive, tape drive)
 - ❑ Varying properties include access speed, capacity, data-transfer rate, access method (sequential or random)
- ❑ File-System management
 - ❑ Files usually organized into directories
 - ❑ Access control on most systems to determine who can access what
 - ❑ OS activities include
 - ❑ **Creating** and **deleting** files and directories
 - ❑ **Primitives** to **manipulate** files and dirs
 - ❑ **Mapping** files onto secondary storage
 - ❑ **Backup** files onto stable (non-volatile) storage media

Mass-Storage Management

- ❑ Usually disks used to store data that does not fit in main memory or data that must be kept for a “long” period of time
- ❑ Proper management is of central importance
- ❑ Entire speed of computer operation hinges on **disk subsystem** and its algorithms
- ❑ OS activities
 - ❑ **Free-space management**
 - ❑ **Storage allocation**
 - ❑ **Disk scheduling**
- ❑ Some storage need not be fast
 - ❑ Tertiary storage includes optical storage, magnetic tape
 - ❑ Still must be managed – by OS or applications
 - ❑ Varies between WORM (write-once, read-many-times) and RW (read-write)

I/O Subsystem

? One purpose of OS is to **hide** peculiarities of hardware devices from the user

? I/O subsystem responsible for

? Memory management of I/O including **buffering** (storing data temporarily while it is being transferred), **caching** (storing parts of data in faster storage for performance), spooling (the overlapping of output of one job with input of other jobs)

? General **device-driver interface**

? **Drivers** for specific hardware devices

Protection and Security

❓ **Protection** – any mechanism for controlling access of processes or users to resources defined by the OS

❓ **Security** – defense of the system against internal and external attacks

❓ Huge range, including denial-of-service, worms, viruses, identity theft, theft of service

❓ Systems generally first distinguish among users, to determine who can do what

❓ User identities (**user IDs**, security IDs) include name and associated number, one per user

❓ User ID then associated with all files, processes of that user to determine access control

❓ Group identifier (**group ID**) allows set of users to be defined and controls managed, then also associated with each process, file

Special-Purpose Systems

❑ **Real-time** embedded systems most prevalent form of computers

❑ Vary considerable, special purpose, limited purpose OS, **real-time OS**

❑ **Multimedia** systems

❑ Streams of data must be delivered according to time restrictions

❑ **Handheld** systems

❑ PDAs, smart phones, limited CPU, memory, power

❑ Reduced feature set OS, limited I/O

Wrap Up

- ? Review/tour the key components of a **Computer System**
- ? Learn about and understand an **operating system**
- ? Learn and understand the **role/duties of an operating system**
 - ? **Processes**
 - ? **Memory**
 - ? **Storage**
 - ? **I/O**
 - ? **Protection and Security**